

1 /*

2
3 --> Set interface:

- 4
- 5 • It was introduced in JDK 1.2 version.
- 6 • It is a direct child interface to Collection interface.
- 7 • It is not index based, it able to arrange all the elements on the basis of elements hashcode values.
- 8 • It will not allow duplicate elements.
- 9 • It will not follow insertion order.

10
11 Note: LinkedHashSet will follow insertion order.

- 12
- 13 • It will not follow Sorting order.

14
15 Note: SortedSet, NavigableSet and TreeSet are following Sorting order.

- 16
- 17 • It able to allow only one null value.

18
19 Note: SortedSet, NavigableSet and TreeSet are not allowing even single null value.

20
21
22 --> HashSet:

- 23
- 24 • HashSet is a direct implementation class to Set interface.
- 25 • It was introduced in JDK 1.2 version.
- 26 • It is not index based, it able to arrange all the elements on the basis of elements hashcode values.
- 27 • It will not allow duplicate elements.
- 28 • It will not follow insertion order.
- 29 • It will not follow Sorting order.
- 30 • It able to allow only one null value.
- 31 • Its internal data structure is "Hashtable".
- 32 • Its initial capacity is "16" elements and its initial fill_Ratio is 75%.
- 33 • It is not synchronized.
- 34 • Almost all the methods are not synchronized in HashSet
- 35 • It allows more than one thread at a time.
- 36 • It follows parallel execution.
- 37 • It will reduce execution time.
- 38 • It improves performance of the applications.
- 39 • It is not giving guarantee for data consistency.
- 40 • It is not threadsafe.

41
42 --> Constructors:

43
44 1. public HashSet():

45 --> This constructor can be used to create an empty HashSet object with 16 elements as initial capacity and 75% fill ratio.

46 EX: HashSet hs = new HashSet();

47
48 2. public HashSet(int capacity):

49 --> This constructor can be used to create an empty HashSet object with the specified capacity as initial capacity

```

50         and with the default fill ratio 75%.
51         EX: HashSet hs = new HashSet(20);
52 3. public HashSet(int capacity, float fill_Ratio):
53 --> This constructor can be used to create an empty HashSet object with
the specified capacity
54         and with the specified fill ratio.
55         EX: HashSet hs = new HashSet(20,0.85f);
56 4. public HashSet(Collection c)
57 --> This constructor can be used to create HashSet object with all the
elements of the specified Collection.
58
59 */
60
61 import java.util.ArrayList;
62 import java.util.HashSet;
63 import java.util.Iterator;
64
65 public class Collection9 {
66     public static void main(String[] args) {
67
68         // Constructors:
69         // 1. boolean add (Element e):
70
71         ArrayList all = new ArrayList<>();
72         all.add(10);
73         all.add(20);
74         all.add(20);
75         all.add(10);
76         all.add(30);
77
78
79         HashSet hs1 = new HashSet<>();
80         System.out.println("10 added ? "+hs1.add(10));
81         hs1.add(20);
82         hs1.add(30);
83         System.out.println("10 added again? "+hs1.add(10));
84         hs1.add(40);
85         hs1.add(50);
86
87         System.out.println(hs1);
88
89         HashSet hs2 = new HashSet<>(10, 0.75f);
90
91         HashSet hs3 = new HashSet<>(5);
92
93         HashSet hs4 = new HashSet<>(all);
94         System.out.println(hs4);
95
96         // 2. boolean remove(Object o):
97
98         System.out.println(hs1.remove(10));
99         System.out.println(hs1.remove(100));
100
101         System.out.println(hs1);
102

```

```
103         // 3. void clear():
104
105         System.out.println(hs1.isEmpty());
106         System.out.println(hs2.isEmpty());
107
108         // 4. boolean addAll (Collection c):
109
110         hs2.addAll(a11);
111
112         System.out.println(hs2);
113
114         // 5. boolean removeAll (Collection c):
115
116         hs1.removeAll(a11);
117         System.out.println(hs1);
118
119         // 6. boolean equals (Object o):
120
121         System.out.println("a11 is: " + a11);
122         System.out.println("hs2 is: "+hs2);
123         System.out.println("hs4 is: "+hs4);
124
125         System.out.println(hs2.equals(a11));
126         System.out.println(hs2.equals(hs4));
127
128         // 7. int size():
129
130         System.out.println(hs1.size());
131         System.out.println(hs2.size());
132
133         // 8. Iterator iterator():
134
135         System.out.println("Use of Iterator: ");
136
137         Iterator itr = hs2.iterator();
138         while (itr.hasNext()) {
139             System.out.println(itr.next());
140         }
141     }
142 }
```